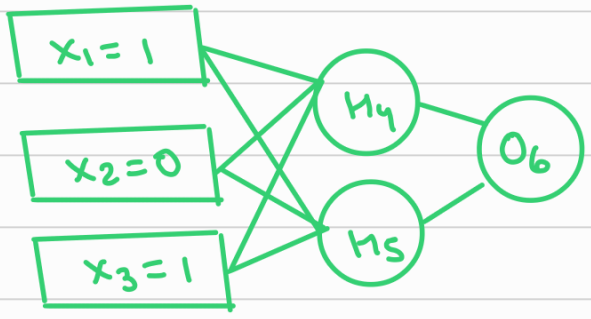
NEURAL N/WS

Assume that neurons have a sigmoid activation function, perform a forward pass & a backward pass on the n/w. Assume that the actual o/p of y is 1 & learning rate is 0.9. Perform another forward pass



$w_{14} = 0.2$
$w_{15} = -0.3$
$w_{24} = 0.4$
$w_{25} = 0.1$
$w_{34} = -0.5$
$w_{35} = 0.2$
$w_{46} = -0.3$
$w_{56} = -0.2$

$\theta_4 = -0.4$
$\theta_5 = 0.2$
$\theta_6 = 0.1$

where θ is bias & w is weight

* Bias tells us how high weighted sum needs to be before neuron starts getting meaningfully active

FORWARD PASS

Compute o/p for y_4, y_5, y_6 (o/p of neurons 4, 5, 6)

$\{a_j = \sum_i (w_{ij} * x_i)\}$, $\{y_j = f(a_j) = 1/(1+e^{-a_j})\}$ → sigmoid activn function

$a_4 = (w_{14} * x_1) + (w_{24} * x_2) + (w_{34} * x_3) + \theta_4$

$= (0.2 * 1) + (0.4 * 0) + (-0.5 * 1) + (-0.4) = -0.7$

$O(H_4) = y_4 = f(a_4) = 1/(1+e^{0.7}) = 0.332$

$a_5 = (w_{15} * x_1) + (w_{25} * x_2) + (w_{35} * x_3) + \theta_5$

$= (-0.3 * 1) + (0.1 * 0) + (0.2 * 1) + 0.2 = 0.1$

$O(H_5) = y_5 = f(a_5) = 1/(1+e^{-0.1}) = 0.525$

$a_6 = (w_{46} * H_4) + (w_{56} * H_5) + \theta_6$

$= (-0.3 * 0.332) + (-0.2 * 0.525) + 0.1 = -0.105$

$O(O_6) = y_6 = f(a_6) = 1/(1+e^{0.105}) = 0.475$

Actual o/p of n/w is 1 & the o/p of our current n/w is 0.475,

Error = $y_{target} - y_6 = 0.526$

BACKWARD PASS

We need to modify the weights of our nn such that we can reduce the error. Each weight is changed by:

$\{\Delta w_{ji} = \eta \delta_j o_i\}$

$\{\delta_j = o_j(1-o_j)(t_j - o_j)\}$ if j is an o/p unit

$\{\delta_j = o_j(1-o_j)\sum_k \delta_k w_{kj}\}$ if j is a hidden unit

where η is a constant called learning rate

t_j is the correct teacher o/p for unit j

δ_j is the error measure for unit j

Compute $\delta_4, \delta_5, \delta_6$

For output unit:

$\delta_6 = y_6(1-y_6)(y_{target} - y_6)$

$= 0.474 * (1-0.474) * (1-0.474) = 0.1311$

For hidden unit:

$\delta_5 = y_5(1-y_5) w_{56} * \delta_6$

$= 0.525(1-0.525) * (-0.2 * 0.1311) = -0.0065$

$\delta_4 = y_4(1-y_4) w_{46} * \delta_6$

$= 0.332(1-0.332) * (-0.3 * 0.1311) = -0.0087$

Compute new weights

$\{\Delta w_{ji} = \eta \delta_j o_i\}$

$\Delta w_{46} = \eta \delta_6 y_4 = 0.9 * 0.1311 * 0.332 = 0.03917$

$w_{46}(new) = \Delta w_{46} + w_{46}(old) = 0.03917 + (-0.3) = -0.261$

$\Delta w_{14} = \eta \delta_4 x_1 = 0.9 * -0.0087 * 1 = -0.0078$

$w_{14}(new) = \Delta w_{14} + w_{14}(old) = -0.0078 + 0.2 = 0.192$

Similarly, update all other weights:

i	j	w_{ij}	δ_j	x_i	η	Updated w_{ij}
4	6	-0.3	0.1311	0.332	0.9	-0.261
5	6	-0.2	0.1311	0.525	0.9	-0.138
1	4	0.2	-0.0087	1	0.9	0.192
1	5	-0.3	-0.0065	1	0.9	-0.306
2	4	0.4	-0.0087	0	0.9	0.4
2	5	0.1	-0.0065	0	0.9	0.1
3	4	-0.5	-0.0087	1	0.9	-0.508
3	5	0.2	-0.0065	1	0.9	0.194

Similarly, update bias weights

θ_j	Previous θ_j	δ_j	η	Updated θ_j
θ_6	0.1	0.1311	0.9	0.218
θ_5	0.2	-0.0065	0.9	0.194
θ_4	-0.4	-0.0087	0.9	-0.408

Forward pass: compute o/p for y_4, y_5, y_6 for updated wts & biases

$$a_j = \sum_i (w_{ij} * x_i), \quad y_j = f(a_j) = 1/(1+e^{-a_j})$$

$$a_4 = (w_{14} * x_1) + (w_{24} * x_2) + (w_{34} * x_3) + \theta_4$$

$$= (0.192 * 1) + (0.4 * 0) + (-0.508 * 1) + (-0.408) = -0.724$$

$$O(H_4) = y_4 = f(a_4) = 1/(1+e^{0.724}) = 0.327$$

$$a_5 = (w_{15} * x_1) + (w_{25} * x_2) + (w_{35} * x_3) + \theta_5$$

$$= (-0.306 * 1) + (0.1 * 0) + (0.194 * 1) + 0.194 = 0.082$$

$$O(H_5) = y_5 = f(a_5) = 1/(1+e^{-0.082}) = 0.520$$

$$a_6 = (w_{46} * H_4) + (w_{56} * H_5) + \theta_6$$

$$= (-0.261 * 0.327) + (-0.138 * 0.520) + 0.218 = 0.061$$

$$O(O_6) = y_6 = f(a_6) = 1/(1+e^{-0.061}) = 0.515 \text{ (new o/p)}$$

Now, Error = $y_{target} - y_6 = 0.485$ ⇒ reduced error

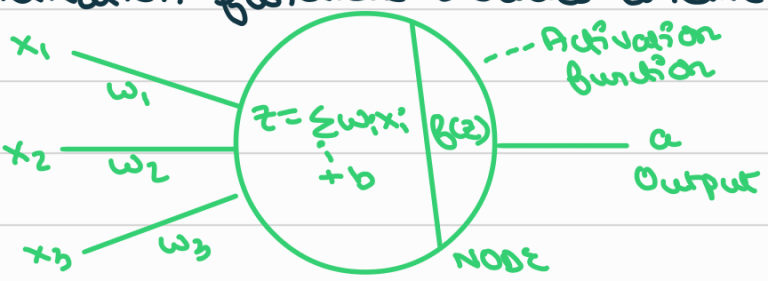
ACTIVATION FUNCTIONS

NNs are comprised of nodes & layers containing an i/p layer, one or more hidden layers & an o/p layer. Each node connects to another & has an associated weight & threshold.

→ If o/p of individual node is above threshold value, that node is activated sending data to next layer of n/w

→ Otherwise, no data is passed along to next layer of n/w

Activation functions decides whether a neuron should be activated.



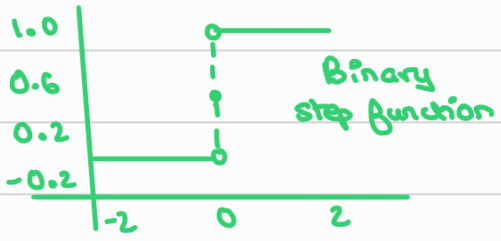
Decides whether neurons i/p to n/w is imp or not

TYPES OF NEURAL N/WS ACTIVATION FUNCTIONS

① Binary step function

→ Depends on a threshold value deciding whether neuron shud be activated or not (i.e., whether o/p is passed to nxt hidden layer)

→ The weighted sum is fed to activa^ func^ & compared to threshold ; if weighted sum greater than it, neuron is activated



Binary step

$$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$$

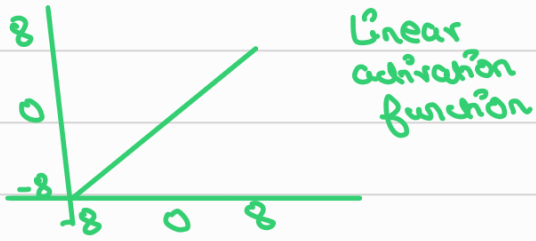
→ Limitations: cannot provide multi value o/ps eg: cant be used for multi^ class classifica^ problems (∵ o/p is 0 or 1)

→ Gradient of step function is 0 ⇒ no backpropagation

② Linear activation function

→ AKA "no activa" or "identity function", is where activation is proportional to i/p.

→ The function doesnt do anything to weighted sum of i/p, it simply spits out val it was given



$$f(x) = x$$

→ Limitations: Derivative of function ($y = mx + b$) is a constant i.e., has no relation to i/p x. Cant be used for backpropaga^

→ All layers of nn will collapse to 1 ∵ no matter no. of layers in nn, last layer will always be linear func^ of 1st layer

→ Hence, model cannot create complex mappings b/w the n/w's i/ps & o/ps

③ Non linear activation functions

→ Allow backpropagation ∵ derivative function is related to i/p & its possible to go back & update wts in i/p neurons for better predictⁿ.

→ Allows stacking of multiple layers of neurons as o/p is non linear combinⁿ of i/p passed thru multiple layers.

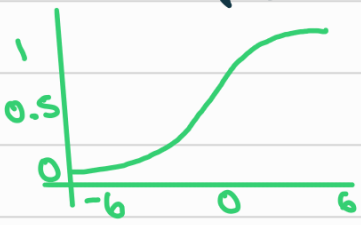
→ Types: Sigmoid, Tanh, ReLU, Leaky ReLU, Parametric ReLU, ELU, Softmax, Swish, GELU, SELU

TYPES OF ACTIVATION FUNCTIONS

① Sigmoid / Logistic activation function

→ This function takes any real value as i/p & o/p vals in range 0 to 1.

→ The larger the i/p (more +ve), the closer the o/p will be to 1. The smaller the i/p (more -ve), closer the o/p will be to 0.



Sigmoid or Logistic (S shaped)

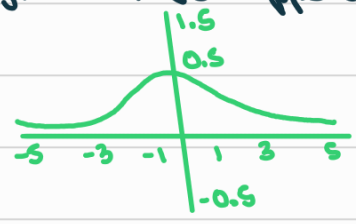
$$f(x) = \frac{1}{1+e^{-x}}$$

→ Widely used for models where we have to predict probability (→ of anything) as o/p. Since probability $\in [0,1]$, sigmoid is good cos of its range

→ Function is differentiable & provides smooth gradient i.e., prevents jumps in o/p vals.

→ Limitations: Gradient vals are only significant in a certain range. Graph gets flatter in other regions

→ In the other flatter regions, the function has very small gradients. As gradient val approaches zero, n/w ceases to learn ⇒ VANISHING GRADIENT

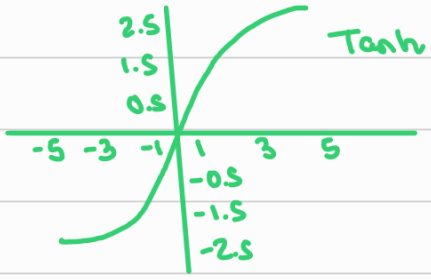


↓ in changes in gradients ⇒
↓ in wt. changes ⇒ learning becomes very slow

② Tanh function (hyperbolic tangent)

→ Similar to sigmoid / logistic activⁿ funcⁿ, has same S shape w/ o/p range of −1 to 1

→ The larger the i/p (more +ve), closer the o/p val will be to 1, whereas smaller the i/p (more -ve), closer the o/p will be to -1.



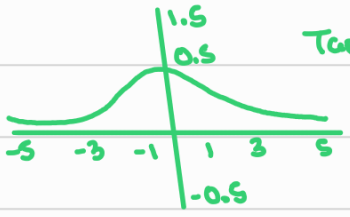
$$f(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$$

→ Advantages: o/p is zero centred i.e., we can easily map o/p vals as strongly -ve, neutral, or strongly +ve

→ Used in hidden layers of NN as vals lie b/w -1 to 1; ∴ mean of hidden layer comes out to be 0 or very close to it

→ Helps in centering data & makes learning for next layer easier.

→ Limitations: It also faces problem of vanishing gradient



$$f'(x) = g(x) = 1 - \tanh^2(x)$$

③ Softmax Function

→ The sigmoid activ^n func^n calculates probability val in range of 0 to 1.

→ However, sum of all classes / o/p probabilities shud be equal to 1. Eg: cant be used if we have 5 o/p vals 0.8, 0.9, 0.7, 0.8, 0.6

→ Softmax activ^n func^n is used. It is a combin^n of multiple sigmoids. It calculates relative probabilities of each sigmoid activ^n func^n in NN.

$$\left[\text{Softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}\right]$$

softmax

where z_i is sigmoid activ^n func^n at a particular neuron

$\sum_j$ → all sigmoids in NN

→ Returns probability of each class. Add all probabilities - sum = 1

→ Used as activ^n func^n for last layer of NN in multi class classification

→ Eg: Assume 3 classes ⇒ 3 neurons in o/p layer. Suppose o/p from neurons is [1.8, 0.9, 0.68]

→ Applying softmax activ^n func^n over these vals to get relative probabilities we get, [0.58, 0.23, 0.19]

↳ sum = 1

→ The function returns 1 for largest probability index & 0 for other two array indexes. Outcome: [0.58, 0.23, 0.19]. Hence index 0 is on.
(1) (0)

④ ReLU function

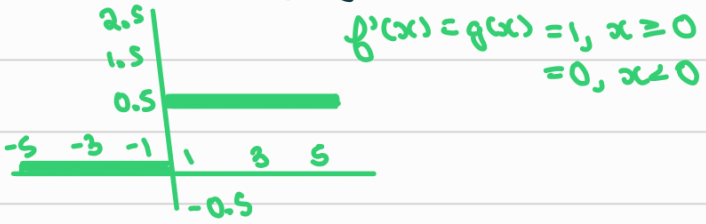
→ ReLU stands for rectified linear unit

→ has a derivative function & allows for backpropaga^n whilst being computationally efficient

→ ReLU function doesn't activate all neurons at same time ⇒ computationally efficient

→ Accelerates the convergence of gradient descent towards the global minimum of the loss func^ due to its linear, non saturating property.

→ Limitations: Dying ReLU problem.



→ The negative side of the graph makes the gradient value zero.

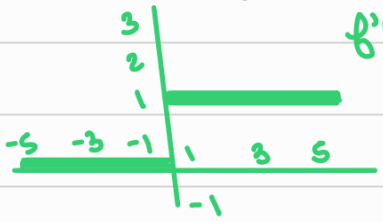
→ Hence, during backpropaga^, wts & biases for some neurons are not updated leading to dead neurons which never get activated

⑤ Leaky ReLU function

→ Solves dying ReLU problem by having a small +ve slope in the -ve area.

→ Advantages: Same as ReLU, enables backpropaga^ even for -ve i/p vals

→ Derivative of leaky ReLU function



$f'(x) = g(x) = 1, \; x \geq 0$
$= 0.01, \; x < 0$

Leaky ReLU (derivative)

→ Limitations: predictⁿ may not be consistent for -ve ip vals ∵ of small +ve slope.

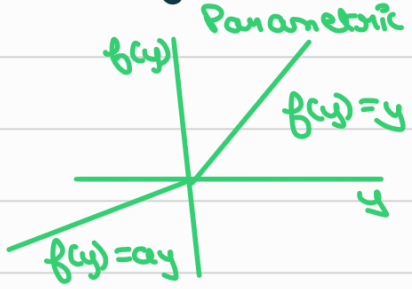
→ All neurons are active unlike ReLU ⇒ time consuming & inefficient

⑥ Parametric ReLU function

→ Variant of ReLU that solves dying ReLU problem too.

→ Provides slope of -ve part of function as an argument a

→ By performing backpropagⁿ, most appropriate val of a is learnt



$f(x) = \max(ax, x)$
where a is slope param for negative values

→ Limitations: may perform differently for diff problems depending upon value of slope param a